PYTHON

ACTIVITY-BASED LEARNING RECORD

Complete documentation for 10 Python programs

Name:	_____
Register Number:	_____
Branch / Department:	_____
Semester:	_____
Subject:	Python Programming

Prepared according to the requested record format: Problem Statement, Objectives, Assumptions, Algorithm, Flowchart, Pseudocode, Python Program, Test Cases, Sample Output, and Conclusion.

Contents

1. University Scholarship Allocation

2. Progressive Electricity Billing

3. Employee Payroll and Deduction Analysis

4. Shopping Cart Billing and Tax Calculation

5. Student Attendance Eligibility Calculator

6. Bank Transaction Validation

7. Timetable Conflict Detection

8. Student Marks and Class Performance Analysis

9. Product Inventory Validation and Valuation

10. Run-Length String Encoding

PROGRAM 1: University Scholarship Allocation

1. Problem Statement

Develop a program to evaluate scholarship applicants using aggregate marks, minimum subject marks, attendance, and family income. Display only eligible applicants and rank them according to academic performance, attendance, and income.

2. Objectives

1. To store applicant details using tuples in a list.

2. To check all scholarship eligibility conditions using if statements.

3. To sort eligible applicants according to the given ranking rules.

4. To display the final rank list.

3. Assumptions

• The data for each applicant is entered in the form (name, aggregate, minimum subject mark, attendance, income).

• Eligibility requires aggregate marks of at least 85, minimum subject marks of at least 75, attendance of at least 90%, and income not exceeding Rs. 3,00,000.

• Ties are resolved by higher attendance and then lower income.

• The sample data is fixed in the program.

4. Algorithm

1. Start the program.

2. Create a list of applicant records and an empty eligible list.

3. Visit each applicant and unpack the tuple values.

4. Check all four eligibility conditions.

5. If every condition is true, append the applicant to the eligible list.

6. Sort eligible applicants by aggregate (descending), attendance (descending), and income (ascending).

7. Display the rank, name, aggregate, attendance, and income.

8. Stop.

5. Flowchart

START | Read applicant records | Check each applicant's four criteria | Eligible? ---- No ----> Skip applicant | Yes Add to eligible list | Sort by marks, attendance, income | Display ranked list | STOP

6. Pseudocode

```
START
  applicants = sample applicant tuples
  eligible = empty list
  FOR each applicant
      IF aggregate >= 85 AND minimum_subject >= 75
         AND attendance >= 90 AND income <= 300000
          ADD applicant to eligible
  SORT eligible by aggregate descending,
       attendance descending, income ascending
  DISPLAY ranked eligible applicants
STOP
```

7. Python Program

```
applicants = [("Asha",91,78,94,250000), ("Bala",88,80,92,280000),
              ("Charan",84,82,96,200000), ("Divya",91,76,94,240000),
              ("Eshan",90,70,98,180000)]
```

```
eligible = []

for name, aggregate, min_subject, attendance, income in applicants:
    if aggregate >= 85 and min_subject >= 75 and attendance >= 90 and income <= 300000:
        eligible.append((name, aggregate, attendance, income))

eligible.sort(key=lambda x: (-x[1], -x[2], x[3]))

print("Rank Name Aggregate Attendance Income")
for rank, (name, aggregate, attendance, income) in enumerate(eligible, 1):
    print(rank, name, aggregate, attendance, income)
```

8. Test Cases

Test case / input	Condition	Expected result
Asha	91, 78, 94, 250000	Eligible
Charan	84, 82, 96, 200000	Not eligible: aggregate below 85
Eshan	90, 70, 98, 180000	Not eligible: minimum subject mark below 75

9. Sample Output

Rank Name Aggregate Attendance Income 1 Divya 91 94 240000 2 Asha 91 94 250000 3 Bala 88 92 280000

10. Conclusion

The program checks scholarship conditions and produces a ranked list of eligible applicants. It demonstrates tuples, lists, conditional statements, and sorting with a key function.

PROGRAM 2: Progressive Electricity Billing

1. Problem Statement

Write a program to calculate electricity bills for several months using progressive unit slabs, fixed charges, a surcharge, and a rebate for low consumption. Compare each bill with the previous month and display a warning if the bill increases by more than 30%.

2. Objectives

1. To calculate energy charges using progressive slabs.

2. To add fixed charges and a 5% surcharge.

3. To apply a Rs. 30 rebate when consumption is below 100 units.

4. To compare consecutive bills and identify a rise greater than 30%.

3. Assumptions

• Monthly consumption values are 80, 120, 190, and 280 units.

• The first 100 units cost Rs. 1.50 per unit.

• Units from 101 to 200 cost Rs. 2.50 per unit; units above 200 cost Rs. 4 per unit.

• A fixed charge of Rs. 50 and surcharge of 5% of energy charges are applied.

• A Rs. 30 rebate is applied when units are below 100.

4. Algorithm

1. Start and initialize the monthly unit list and previous bill as None.

2. For each month, calculate energy charges using the applicable slab or slabs.

3. Add Rs. 50 fixed charge and 5% of energy charges.

4. If consumption is below 100 units, subtract Rs. 30.

5. Display month, units, and bill.

6. If a previous bill exists and the increase is greater than 30%, display a warning.

7. Store the current bill as the previous bill.

8. Stop after all months are processed.

5. Flowchart

START | Read monthly unit list | For each month, calculate slab charges | Add fixed charge and surcharge | Units below 100? -- Yes --> Apply rebate | No Compare with previous bill (if available) | Increase > 30%? -- Yes --> Print warning | Save current bill; next month | STOP

6. Pseudocode

```
START
  monthly_units = [80, 120, 190, 280]
  previous_bill = NONE
  FOR each month and units
      CALCULATE energy using progressive slabs
      bill = energy + 50 + energy * 0.05
      IF units < 100: bill = bill - 30
      DISPLAY month, units, bill
      IF previous_bill exists AND increase > 30%
          DISPLAY warning
      previous_bill = bill
STOP
```

7. Python Program

```
monthly_units = [80, 120, 190, 280]
```

```
previous_bill = None

for month, units in enumerate(monthly_units, 1):
    if units <= 100:
        energy = units * 1.50
    elif units <= 200:
        energy = 150 + (units - 100) * 2.50
    else:
        energy = 150 + 250 + (units - 200) * 4

    bill = energy + 50 + energy * 0.05
    if units < 100:
        bill -= 30

    print("Month", month, "Units:", units, "Bill: Rs.", round(bill, 2))
    if previous_bill is not None and (bill - previous_bill) / previous_bill * 100 > 30:
        print("Warning: bill increased by more than 30%")
    previous_bill = bill
```

8. Test Cases

Test case / input	Condition	Expected result
Month 1	80 units	Rs. 146.00 after rebate
Month 2	120 units	Rs. 266.00
Month 4	280 units	Rs. 959.00; increase warning compared with month 3

9. Sample Output

Month 1 Units: 80 Bill: Rs. 146.0 Month 2 Units: 120 Bill: Rs. 260.0 Month 3 Units: 190 Bill: Rs. 443.75 Warning: bill increased by more than 30% Month 4 Units: 280 Bill: Rs. 959.0 Warning: bill increased by more than 30%

10. Conclusion

The program calculates monthly electricity bills with progressive rates and detects unusually large increases. It uses loops, if-elif-else statements, arithmetic, and comparison.

PROGRAM 3: Employee Payroll and Deduction Analysis

1. Problem Statement

Develop a payroll program that calculates an employee's gross salary from basic pay, allowances, overtime, and incentive. Calculate deductions using progressive deduction slabs and display the gross salary, deduction, and net salary.

2. Objectives

1. To calculate gross salary from its components.

2. To apply the appropriate deduction slab.

3. To calculate net salary after deduction.

4. To print a clear salary statement.

3. Assumptions

• Basic pay is Rs. 22,000, allowances Rs. 5,000, overtime Rs. 1,800, and incentive Rs. 1,200.

• Gross salary is the sum of all four components.

• No deduction is applied up to Rs. 15,000.

• The amount from Rs. 15,001 to Rs. 25,000 is deducted at 5%; the amount above Rs. 25,000 is deducted at 10%, in addition to the first slab deduction.

4. Algorithm

1. Start and assign basic pay, allowances, overtime, and incentive.

2. Calculate gross salary by adding all components.

3. If gross salary is at most Rs. 15,000, set deduction to zero.

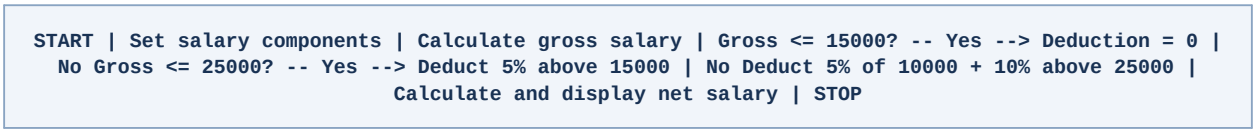
4. If gross salary is up to Rs. 25,000, deduct 5% of the amount above Rs. 15,000.

5. Otherwise, deduct 5% of the first Rs. 10,000 slab and 10% of the amount above Rs. 25,000.

6. Calculate net salary as gross salary minus deduction.

7. Display salary details and stop.

5. Flowchart



6. Pseudocode

```
START
  basic, allowances, overtime, incentive = given values
  gross = sum of salary components
  IF gross <= 15000: deduction = 0
  ELSE IF gross <= 25000:
      deduction = (gross - 15000) * 0.05
  ELSE:
      deduction = 10000 * 0.05 + (gross - 25000) * 0.10
  net = gross - deduction
  DISPLAY gross, deduction, net
STOP
```

7. Python Program

```
basic, allowances, overtime, incentive = 22000, 5000, 1800, 1200
gross = basic + allowances + overtime + incentive

if gross <= 15000:
```

```
    deduction = 0
elif gross <= 25000:
    deduction = (gross - 15000) * 0.05
else:
    deduction = 10000 * 0.05 + (gross - 25000) * 0.10

print("Basic pay:", basic, "Allowances:", allowances,
      "Overtime:", overtime, "Incentive:", incentive)
print("Gross salary:", gross, "Deduction:", round(deduction, 2),
      "Net salary:", round(gross - deduction, 2))
```

8. Test Cases

Test case / input	Condition	Expected result
Gross Rs. 14,000	Below first threshold	Deduction Rs. 0
Gross Rs. 20,000	Within 5% slab	Deduction Rs. 250
Given salary data	Gross Rs. 30,000	Deduction Rs. 1,500; net Rs. 28,500

9. Sample Output

Basic pay: 22000 Allowances: 5000 Overtime: 1800 Incentive: 1200 Gross salary: 30000 Deduction: 1500.0 Net salary: 28500.0

10. Conclusion

The program prepares a basic salary statement and applies progressive deductions correctly. It demonstrates variables, arithmetic operations, and conditional statements.

PROGRAM 4: Shopping Cart Billing and Tax Calculation

1. Problem Statement

Create a shopping cart billing program that calculates each item's line amount, total subtotal, a discount subject to a maximum limit, item-wise tax after allocating the discount proportionally, and the final payable amount.

2. Objectives

1. To store product details in tuples.

2. To calculate subtotal from quantity and price.

3. To apply a percentage discount with a maximum cap.

4. To calculate tax on the discounted value of each item and display the grand total.

3. Assumptions

• Notebook: price Rs. 80, quantity 10, tax rate 5%.

• Pen Pack: price Rs. 120, quantity 5, tax rate 12%.

• Bag: price Rs. 900, quantity 1, tax rate 18%.

• A 10% discount is offered when subtotal is at least Rs. 2,000; discount is capped at Rs. 500.

• The total discount is allocated to each item in proportion to its line amount.

4. Algorithm

1. Start and create a list of item tuples.

2. Calculate subtotal by adding price multiplied by quantity for every item.

3. If subtotal is at least Rs. 2,000, calculate 10% discount, limited to Rs. 500; otherwise discount is zero.

4. For each item, calculate its line amount and proportional discount.

5. Calculate tax on line amount minus allocated discount.

6. Add all item taxes to total tax.

7. Display item details, subtotal, discount, tax, and grand total.

8. Stop.

5. Flowchart

START | Read item tuples | Calculate subtotal | Subtotal >= 2000? -- Yes --> Discount = min(10%, Rs. 500) | No Discount = 0 | For each item: allocate discount proportionally | Calculate tax on discounted line amount | Grand total = subtotal - discount + tax | STOP

6. Pseudocode

```
START
  items = product tuples
  subtotal = SUM(price * quantity)
  IF subtotal >= 2000:
      discount = MIN(subtotal * 0.10, 500)
  ELSE: discount = 0
  total_tax = 0
  FOR each item:
      line = price * quantity
      allocated_discount = discount * line / subtotal
      tax = (line - allocated_discount) * tax_rate / 100
      total_tax = total_tax + tax
      DISPLAY item and tax
  DISPLAY subtotal, discount, total_tax,
          subtotal - discount + total_tax
STOP
```

7. Python Program

```
items = [("Notebook",80,10,5), ("Pen Pack",120,5,12), ("Bag",900,1,18)]
subtotal = 0
for name, price, qty, tax_rate in items:
    subtotal += price * qty

discount = min(subtotal * 0.10, 500) if subtotal >= 2000 else 0
total_tax = 0

for name, price, qty, tax_rate in items:
    line = price * qty
    allocated_discount = discount * line / subtotal if subtotal else 0
    tax = (line - allocated_discount) * tax_rate / 100
    total_tax += tax
    print(name, "Qty:", qty, "Line:", line, "Tax:", round(tax, 2))

print("Subtotal:", subtotal, "Discount:", discount,
      "Tax:", round(total_tax, 2),
      "Grand total:", round(subtotal - discount + total_tax, 2))
```

8. Test Cases

Test case / input	Condition	Expected result
Notebook	80 x 10	Line amount Rs. 800
Pen Pack	120 x 5	Line amount Rs. 600
Complete cart	Subtotal Rs. 2,300	Discount Rs. 230; tax Rs. 184.90; grand total Rs. 2,254.90

9. Sample Output

Notebook Qty: 10 Line: 800 Tax: 38.0 Pen Pack Qty: 5 Line: 600 Tax: 63.6 Bag Qty: 1 Line: 900 Tax: 83.3 Subtotal: 2300 Discount: 230.0 Tax: 184.9 Grand total: 2254.9

10. Conclusion

The program calculates a shopping bill with a capped discount and item-wise tax. It demonstrates lists of tuples, loops, arithmetic, and conditional expressions.

PROGRAM 5: Student Attendance Eligibility Calculator

1. Problem Statement

Write a program to calculate a student's current attendance percentage and determine the minimum number of additional classes the student must attend consecutively to reach the required attendance percentage.

2. Objectives

1. To validate attendance input.

2. To calculate current attendance percentage.

3. To use a while loop to find the required additional classes.

4. To display the projected attendance percentage.

3. Assumptions

• Classes attended = 42; total classes = 60; target attendance = 75%.

• Attendance values cannot be negative and attended classes cannot exceed total classes.

• The student attends every additional class; no new absence occurs.

• The target is considered achieved when projected attendance is equal to or greater than 75%.

4. Algorithm

1. Start and initialize attended classes, total classes, and target percentage.

2. Check whether the attendance data is valid.

3. If invalid, display an error message.

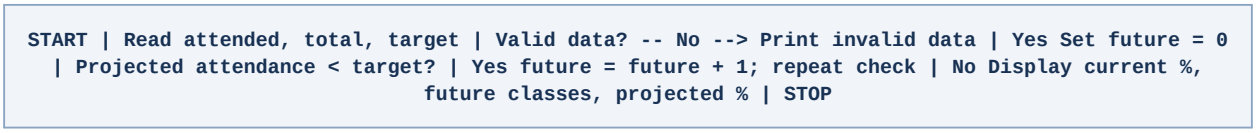
4. Otherwise, initialize future classes to zero.

5. While projected attendance is below the target, increase future classes by one.

6. Display current attendance, additional classes required, and projected attendance.

7. Stop.

5. Flowchart



6. Pseudocode

```
START
  attended = 42; total = 60; target = 75
  IF total <= 0 OR attended < 0 OR attended > total
      DISPLAY invalid attendance data
  ELSE
      future = 0
      WHILE ((attended + future) / (total + future)) * 100 < target
          future = future + 1
      DISPLAY current percentage
      DISPLAY future classes required
      DISPLAY projected percentage
STOP
```

7. Python Program

```
attended, total, target = 42, 60, 75

if total <= 0 or attended < 0 or attended > total:
    print("Invalid attendance data.")
else:
    future = 0
```

```
while (attended + future) / (total + future) * 100 < target:
    future += 1

print("Current attendance:", round(attended / total * 100, 2), "%")
print("Additional classes required:", future)
print("Projected attendance:",
      round((attended + future) / (total + future) * 100, 2), "%")
```

8. Test Cases

Test case / input	Condition	Expected result
42 attended, 60 total	Target 75%	12 additional classes
45 attended, 60 total	Target 75%	0 additional classes
61 attended, 60 total	Invalid input	Invalid attendance data

9. Sample Output

Current attendance: 70.0 % Additional classes required: 12 Projected attendance: 75.0 %

10. Conclusion

The program finds the number of consecutive classes needed to reach the attendance target. It demonstrates input validation, conditional statements, and a while loop.

PROGRAM 6: Bank Transaction Validation

1. Problem Statement

Develop a simple bank transaction program that processes deposits, withdrawals, and transfer requests. Reject invalid amounts, withdrawals above the permitted limit, and withdrawals exceeding the available balance. Display transaction counts and the closing balance.

2. Objectives

1. To process a sequence of transaction tuples.

2. To update the account balance for valid deposits and withdrawals.

3. To reject invalid or unsupported transactions.

4. To display accepted and rejected counts, deposits, withdrawals, and closing balance.

3. Assumptions

• The opening balance is Rs. 5,000.

• Transactions are provided as (transaction type, amount).

• Amounts must be greater than zero.

• A withdrawal is accepted only when it is at most Rs. 3,000 and does not exceed the balance.

• Only DEPOSIT and WITHDRAW are supported by this program; TRANSFER is rejected.

4. Algorithm

1. Start with balance Rs. 5,000 and set all counters to zero.

2. Read each transaction type and amount.

3. If amount is zero or negative, reject it and increase rejected count.

4. If type is DEPOSIT, add the amount to balance and update deposit and accepted counters.

5. Else if type is WITHDRAW and amount is at most Rs. 3,000 and at most the balance, subtract it and update withdrawal and accepted counters.

6. Otherwise reject the transaction.

7. After all transactions, display counters and closing balance.

8. Stop.

5. Flowchart

START | Set opening balance and counters | Read next transaction | Amount <= 0? -- Yes --> Reject | No DEPOSIT? -- Yes --> Add amount to balance | No Valid WITHDRAW (<= 3000 and <= balance)? | Yes --> Subtract amount | No --> Reject | More transactions? -- Yes --> Repeat | No Display summary and closing balance | STOP

6. Pseudocode

```
START
  balance = 5000; counters = 0
  FOR each kind, amount
      IF amount <= 0: reject
      ELSE IF kind is DEPOSIT:
          balance = balance + amount
          update deposits and accepted
      ELSE IF kind is WITHDRAW AND amount <= 3000
              AND amount <= balance:
          balance = balance - amount
          update withdrawals and accepted
      ELSE: reject
  DISPLAY accepted, rejected, deposits, withdrawals, balance
```

```
STOP
```

7. Python Program

```
transactions = [("DEPOSIT",2000), ("WITHDRAW",1500), ("WITHDRAW",4000),
                ("DEPOSIT",0), ("WITHDRAW",1000), ("TRANSFER",500)]
balance, deposits, withdrawals, accepted, rejected = 5000, 0, 0, 0, 0

for kind, amount in transactions:
    if amount <= 0:
        print("Rejected:", kind, amount)
        rejected += 1
    elif kind == "DEPOSIT":
        balance += amount
        deposits += amount
        accepted += 1
    elif kind == "WITHDRAW" and amount <= 3000 and amount <= balance:
        balance -= amount
        withdrawals += amount
        accepted += 1
    else:
        print("Rejected transaction:", kind, amount)
        rejected += 1

print("Accepted:", accepted, "Rejected:", rejected,
      "Deposits:", deposits, "Withdrawals:", withdrawals,
      "Closing balance:", balance)
```

8. Test Cases

Test case / input	Condition	Expected result
DEPOSIT 2000	Positive deposit	Accepted; balance Rs. 7000
WITHDRAW 4000	Above withdrawal limit	Rejected
WITHDRAW 1000	Within limit and balance	Accepted
TRANSFER 500	Unsupported transaction	Rejected

9. Sample Output

Rejected transaction: WITHDRAW 4000 Rejected: DEPOSIT 0 Rejected transaction: TRANSFER 500 Accepted: 3 Rejected: 3 Deposits: 2000 Withdrawals: 2500 Closing balance: 4500

10. Conclusion

The program validates transactions and maintains an updated account balance. It demonstrates tuple unpacking, loops, conditions, counters, and arithmetic.

PROGRAM 7: Timetable Conflict Detection

1. Problem Statement

Create a program to examine a class timetable and identify overlapping class periods. Also report whether two overlapping classes are assigned to the same room.

2. Objectives

1. To store timetable entries as tuples.

2. To compare every pair of classes.

3. To detect time overlap on the same day.

4. To identify room conflicts and count overlapping pairs.

3. Assumptions

• Each tuple contains (course, day, start time, end time, room).

• Two classes overlap if they are on the same day and each starts before the other ends.

• A room conflict occurs when overlapping classes use the same room.

• Times are represented as decimal hours, such as 9.5 for 9:30.

4. Algorithm

1. Start and create the timetable list.

2. Set the conflict counter to zero.

3. Use an outer loop to select one timetable entry.

4. Use an inner loop to compare it with each later entry only once.

5. Unpack both entries.

6. Check whether the day is the same and the time intervals overlap.

7. If they overlap, display the two courses and increase the counter.

8. If their room numbers are equal, display a room conflict.

9. After all comparisons, display that the timetable is valid if no overlap exists; otherwise display the number of overlapping pairs.

10. Stop.

5. Flowchart

START | Read timetable entries | Compare each pair of entries | Same day and overlapping times? | No --> Check next pair | Yes Display time conflict; count it | Same room? -- Yes --> Display room conflict | More pairs? -- Yes --> Repeat | No Any conflicts? --> Display summary | STOP

6. Pseudocode

```
START
  schedule = timetable tuples
  conflicts = 0
  FOR i from 0 to length(schedule)-1
      FOR j from i+1 to length(schedule)-1
          unpack schedule[i] and schedule[j]
          IF day1 == day2 AND start1 < end2 AND start2 < end1
              DISPLAY time conflict
              conflicts = conflicts + 1
              IF room1 == room2: DISPLAY room conflict
  IF conflicts == 0: DISPLAY timetable is valid
  ELSE: DISPLAY number of overlapping pairs
STOP
```

7. Python Program

```
schedule = [("Python","MON",9.0,10.0,"R101"),
            ("Maths","MON",9.5,10.5,"R102"),
            ("Physics","MON",10.0,11.0,"R101"),
            ("English","TUE",9.0,10.0,"R101")]
conflicts = 0

for i in range(len(schedule)):
    course1, day1, start1, end1, room1 = schedule[i]
    for j in range(i + 1, len(schedule)):
        course2, day2, start2, end2, room2 = schedule[j]
        if day1 == day2 and start1 < end2 and start2 < end1:
            print("Time conflict:", course1, "and", course2, "on", day1)
            conflicts += 1
            if room1 == room2:
                print("Room conflict:", room1)

if conflicts == 0:
    print("Timetable is valid.")
else:
    print("Overlapping pairs:", conflicts)
```

8. Test Cases

Test case / input	Condition	Expected result
Python 9:00-10:00, Maths 9:30-10:30	Same day, different rooms	Time overlap
Python and Physics	Same day, same room R101; boundary at 10:00	No overlap because one ends when the other starts
English Tuesday	Other classes Monday	No conflict

9. Sample Output

Time conflict: Python and Maths on MON Overlapping pairs: 1

10. Conclusion

The program compares timetable intervals and reports overlapping classes and shared-room conflicts. With the supplied data, only Python and Maths overlap; the Physics class begins exactly when Python ends, so it is not counted as an overlap.

PROGRAM 8: Student Marks and Class Performance Analysis

1. Problem Statement

Write a program to calculate each student's total and average marks, identify the student with the highest total, and calculate the average mark for each subject. Display a warning when a subject's class average is below 50.

2. Objectives

1. To store student names, subjects, and marks in lists.

2. To calculate total and average marks for each student.

3. To identify the top-performing student.

4. To calculate subject-wise class averages and show warnings.

3. Assumptions

• There are four students and three subjects: Python, Maths, and Physics.

• Marks are stored in a nested list; each inner list contains one student's subject marks in subject order.

• All subjects have equal weight.

• A subject average below 50 produces a warning.

4. Algorithm

1. Start and create student, subject, and marks lists.

2. Create an empty totals list.

3. For each student's marks, calculate the total and average; append the total and display the result.

4. Find the index of the maximum total and display that student's name and total.

5. For each subject, calculate the average across all students.

6. Display the subject average and print a warning if it is below 50.

7. Stop.

5. Flowchart

START | Read students, subjects, nested marks | For each student: total and average | Find and display highest total | For each subject: calculate class average | Average < 50? -- Yes --> Print warning | No Display subject average | STOP

6. Pseudocode

```
START
  students, subjects, marks = sample data
  totals = empty list
  FOR each student's marks
      total = SUM(marks)
      ADD total to totals
      DISPLAY student, total, total / number of subjects
  top = index of maximum total
  DISPLAY top student and total
  FOR each subject index
      average = SUM(all marks at that index) / student count
      DISPLAY subject and average
      IF average < 50: DISPLAY warning
STOP
```

7. Python Program

```
students = ["Asha","Bala","Charan","Divya"]
subjects = ["Python","Maths","Physics"]
```

```
marks = [[88,76,81],[67,72,58],[45,49,52],[91,85,89]]
totals = []

for i in range(len(marks)):
    total = sum(marks[i])
    totals.append(total)
    print(students[i], "Total:", total,
          "Average:", round(total / len(subjects), 2))

top = totals.index(max(totals))
print("Top performer:", students[top], "Total:", totals[top])

for j, subject in enumerate(subjects):
    average = sum(marks[i][j] for i in range(len(marks))) / len(marks)
    print(subject, "average:", round(average, 2))
    if average < 50:
        print("Warning: low class average")
```

8. Test Cases

Test case / input	Condition	Expected result
Asha: 88, 76, 81	Three subject marks	Total 245; average 81.67
Divya: 91, 85, 89	Three subject marks	Total 265; highest total
Charan's marks in Python	45	Included in subject average

9. Sample Output

Asha Total: 245 Average: 81.67 Bala Total: 197 Average: 65.67 Charan Total: 146 Average: 48.67 Divya Total: 265 Average: 88.33 Top performer: Divya Total: 265 Python average: 72.75 Maths average: 70.5 Physics average: 70.0

10. Conclusion

The program summarizes individual and subject-wise academic performance and identifies the top performer. It demonstrates nested lists, loops, indexing, sum, max, and average calculations.

PROGRAM 9: Product Inventory Validation and Valuation

1. Problem Statement

Develop an inventory program that checks product records for invalid product IDs, negative prices, or negative stock. For valid records, calculate inventory value and display a replenishment message when stock is below 10 units.

2. Objectives

1. To store product records as tuples.

2. To validate each product record.

3. To calculate the value of valid inventory items.

4. To identify products that need replenishment.

3. Assumptions

• Each record contains (product ID, unit price, stock quantity).

• A blank product ID, negative price, or negative stock makes a record invalid.

• Zero price and zero stock are allowed by the shown validation logic.

• A replenishment message is printed when valid stock is below 10 units.

• Inventory value is unit price multiplied by stock.

4. Algorithm

1. Start and create the product tuple list.

2. Initialize total inventory value to zero.

3. For each product, unpack ID, price, and stock.

4. If the ID is blank, price is negative, or stock is negative, display an invalid record message.

5. Otherwise calculate product value and add it to inventory value.

6. Display the product stock and value.

7. If stock is below 10, display a replenishment message.

8. After all records, display total valid inventory value.

9. Stop.

5. Flowchart

START | Read product records | For each product, validate ID, price, stock | Invalid? -- Yes --> Display invalid record | No Calculate price x stock; add to total | Stock < 10? -- Yes --> Display replenishment message | No More records? -- Yes --> Repeat | No Display total inventory value | STOP

6. Pseudocode

```
START
  products = product tuples
  inventory_value = 0
  FOR each product_id, price, stock
      IF product_id is blank OR price < 0 OR stock < 0
          DISPLAY invalid record
      ELSE
          value = price * stock
          inventory_value = inventory_value + value
          DISPLAY product, stock, value
          IF stock < 10: DISPLAY replenishment message
  DISPLAY inventory_value
STOP
```

7. Python Program

```
products = [("P101",50.0,25), ("P102",120,0), ("P103",75.0,0),
            ("",40.0,8), ("P105",-10.0,12)]
inventory_value = 0

for product_id, price, stock in products:
    if product_id == "" or price < 0 or stock < 0:
        print("Invalid record:", product_id, price, stock)
    else:
        value = price * stock
        inventory_value += value
        print(product_id, "Stock:", stock, "Value:", round(value, 2))
        if stock < 10:
            print("Replenish", 10 - stock, "units")

print("Total valid inventory value:", round(inventory_value, 2))
```

8. Test Cases

Test case / input	Condition	Expected result
P101, Rs. 50, stock 25	Valid record	Value Rs. 1,250
P102, Rs. 120, stock 0	Valid but low stock	Value Rs. 0; replenish 10 units
Blank ID, Rs. 40, stock 8	Invalid ID	Record rejected
P105, price -10	Negative price	Record rejected

9. Sample Output

P101 Stock: 25 Value: 1250.0 P102 Stock: 0 Value: 0 Replenish 10 units P103 Stock: 0 Value: 0.0 Replenish 10 units Invalid record: 40.0 8 Invalid record: P105 -10.0 12 Total valid inventory value: 1250.0

10. Conclusion

The program validates inventory data, values valid stock, and flags low-stock items. It demonstrates tuples, loops, conditional validation, and accumulator variables.

PROGRAM 10: Run-Length String Encoding

1. Problem Statement

Write a program to compress a string by replacing each consecutive group of identical characters with the character followed by its repetition count. Compare the encoded and original lengths and display the shorter representation.

2. Objectives

1. To examine a string character by character.

2. To count consecutive repeated characters.

3. To build a run-length encoded string.

4. To compare the encoded length with the original length.

3. Assumptions

• The sample input is the non-empty string AAABBBCCCD.

• Only consecutive identical characters are grouped together.

• The encoded form stores each character followed by its count, including a count of 1.

• The program displays the encoded string only when it is shorter; otherwise it displays the original string.

4. Algorithm

1. Start and assign the input string.

2. If the string is empty, display an empty-input message.

3. Otherwise initialize encoded as an empty string and count as 1.

4. Visit each character from the second character to the end.

5. If the current character equals the previous character, increase count.

6. Otherwise append the previous character and count to encoded, then reset count to 1.

7. After the loop, append the final character and its count.

8. Display original string and length, encoded string and length, and the shorter representation.

9. Stop.

5. Flowchart

START | Read input string | Is string empty? -- Yes --> Display empty message | No Set encoded = empty, count = 1 | Compare current char with previous char | Same? -- Yes --> Increase count | No Append previous char + count; reset count | Append final char + count | Compare lengths and display shorter form | STOP

6. Pseudocode

```
START
  text = input string
  IF text is empty: DISPLAY empty message
  ELSE
      encoded = empty string; count = 1
      FOR i from 1 to length(text)-1
          IF text[i] == text[i-1]: count = count + 1
          ELSE:
              encoded = encoded + text[i-1] + count
              count = 1
      encoded = encoded + last character + count
      DISPLAY original and encoded lengths
      IF encoded length < original length: DISPLAY encoded
      ELSE: DISPLAY original
STOP
```

7. Python Program

```
text = "AAABBBCCCD"

if text == "":
    print("Input string is empty.")
else:
    encoded, count = "", 1
    for i in range(1, len(text)):
        if text[i] == text[i-1]:
            count += 1
        else:
            encoded += text[i-1] + str(count)
            count = 1

    encoded += text[-1] + str(count)
    print("Original:", text, "Length:", len(text))
    print("Encoded:", encoded, "Length:", len(encoded))
    print("Chosen representation:", encoded if len(encoded) < len(text) else text)
```

8. Test Cases

Test case / input	Condition	Expected result
AAABBBCCCD	Repeated groups	A3B3C3D1; encoded length 8
ABCD	No repeated groups	A1B1C1D1; original is shorter
Empty string	No characters	Input string is empty

9. Sample Output

Original: AAABBBCCCD Length: 10 Encoded: A3B3C3D1 Length: 8 Chosen representation: A3B3C3D1

10. Conclusion

The program performs simple run-length encoding and chooses the shorter representation. It demonstrates string indexing, loops, conditions, concatenation, and length comparison.